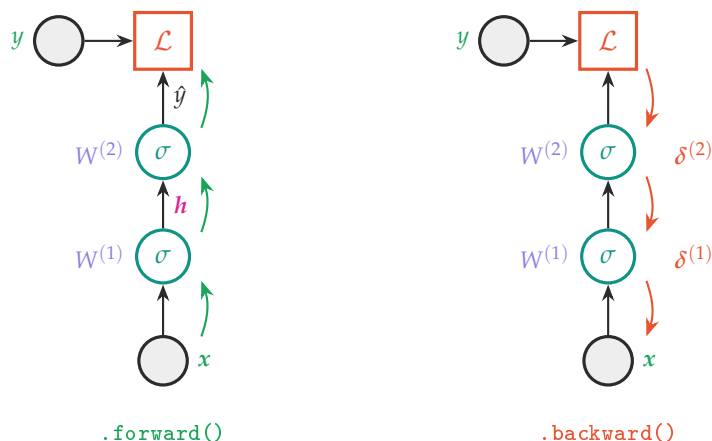


Backpropagation (2)

DS 4440 · LECTURE 6

1. Last time

Gradients on computation graphs via **backprop**. The same graph supports two traversals:



For implementation, each node or layer must know how to go **forward** and **backward**.

Throughout, $\tilde{h}^{(k)} = \mathbf{h}^{(k-1)} W^{(k)}$ is the **pre-activation** of layer k and $\mathbf{h}^{(k)} = \sigma(\tilde{h}^{(k)})$ its output, and

$$\delta^{(k)} \stackrel{\text{def}}{=} \frac{\partial \mathcal{L}}{\partial \tilde{h}^{(k)}}$$

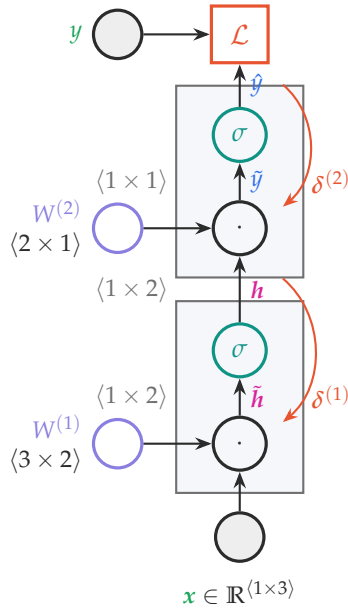
is the node error there — always at the *pre-activation*, as in Lecture 5.

2. An example, end to end

Take binary cross entropy on a two-layer network:

$$\mathcal{L} = \text{BCELoss}(y, \hat{y}) = - \underbrace{y}_{0/1} \log \hat{y} - (1 - y) \log(1 - \underbrace{\hat{y}}_{\text{probability}}),$$

$$\tilde{\mathbf{h}} = \mathbf{x} W^{(1)}, \quad \mathbf{h} = \sigma(\tilde{\mathbf{h}}), \quad \tilde{\mathbf{y}} = \mathbf{h} W^{(2)}, \quad \hat{y} = \sigma(\tilde{y}).$$



Backward, step by step

Start at the top. The output layer's node error sits at $\tilde{y}$:

$$\delta^{(2)} = \frac{\partial \mathcal{L}}{\partial \tilde{y}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \tilde{y}}}_{\text{incoming}} \underbrace{\sigma'(\tilde{y})}_{\text{gate}} \stackrel{\text{BCE}+\sigma}{=} \tilde{y} - y,$$

the cancellation from Lecture 5. Every line below is (incoming) $\times$ (local derivative), with the local factor named underneath:

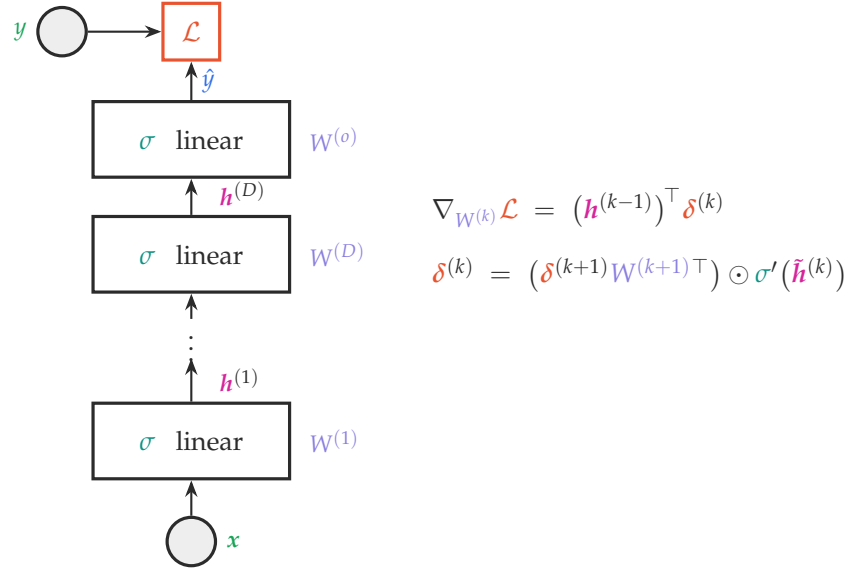
$$\nabla_{W^{(2)}} \mathcal{L} = \underbrace{\tilde{h}^\top}_{\text{local: } \partial \tilde{y} / \partial W^{(2)}} \underbrace{\delta^{(2)}}_{\text{incoming}} \quad \langle 2 \times 1 \rangle \langle 1 \times 1 \rangle = \langle 2 \times 1 \rangle \checkmark$$

$$\delta^{(1)} = \left(\underbrace{\delta^{(2)}}_{\text{incoming}} \underbrace{W^{(2)\top}}_{\text{local (route): } \partial \tilde{y} / \partial \tilde{h}} \right) \odot \underbrace{\sigma'(\tilde{h})}_{\text{local (gate): } \partial \tilde{h} / \partial \tilde{h}} \quad \langle 1 \times 1 \rangle \langle 1 \times 2 \rangle = \langle 1 \times 2 \rangle \checkmark$$

$$\nabla_{W^{(1)}} \mathcal{L} = \underbrace{x^\top}_{\text{local: } \partial \tilde{h} / \partial W^{(1)}} \underbrace{\delta^{(1)}}_{\text{incoming}} \quad \langle 3 \times 1 \rangle \langle 1 \times 2 \rangle = \langle 3 \times 2 \rangle \checkmark$$

Every line is the same two moves: **route** through a weight matrix (no σ' , since $\tilde{h}^{(k)}$ depends on $\tilde{h}^{(k-1)}$ linearly), then **gate** by σ' at the pre-activation. The weight gradient at a layer only ever needs the node error there and the input that arrived from below.

3. Generalizing: a network with D layers



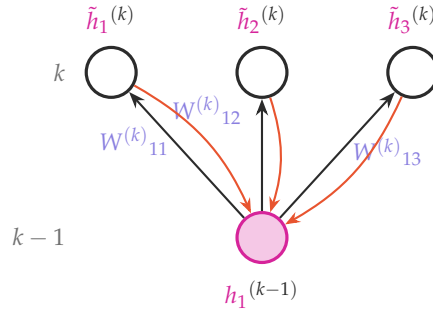
For a general layer, the route step is a **Jacobian** — the matrix of partials of that layer's output with respect to its input:

$$\frac{\partial \tilde{\mathbf{h}}^{(k+1)}}{\partial \mathbf{h}^{(k)}} = \begin{bmatrix} \frac{\partial \tilde{h}_1^{(k+1)}}{\partial h_1^{(k)}} & \cdots & \frac{\partial \tilde{h}_d^{(k+1)}}{\partial h_1^{(k)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial \tilde{h}_1^{(k+1)}}{\partial h_d^{(k)}} & \cdots & \frac{\partial \tilde{h}_d^{(k+1)}}{\partial h_d^{(k)}} \end{bmatrix}$$

For a fully connected layer we never have to build it: $\tilde{\mathbf{h}}^{(k+1)} = \mathbf{h}^{(k)} W^{(k+1)}$, so the Jacobian is $W^{(k+1)}$, and the route step is a matrix multiply. For an activation it is diagonal, and the gate step is a Hadamard product. A custom layer only has to supply its own version of these two.

4. Why the fully connected case is intuitive

Look at one unit, $h_1^{(k-1)}$, and the units it feeds:



So the blame that unit 1 of layer $k-1$ takes for the error at $\tilde{h}_j^{(k)}$ scales with the connection weight

between them. Summing over everything it feeds, and then gating at its own pre-activation:

$$\delta_1^{(k-1)} = \underbrace{\sum_{j=1}^d \underbrace{W_{1j}^{(k)}}_{\text{influence of } h_1^{(k-1)} \text{ on } \tilde{h}_j^{(k)}} \underbrace{\delta_j^{(k)}}_{\text{node error at } j}}_{\partial \mathcal{L} / \partial h_1^{(k-1)}} \cdot \sigma'(\tilde{h}_1^{(k-1)}).$$

Because δ is defined at pre-activations, the sum is exact — no σ' hides inside it, and the single σ' that does appear is attached to this unit's own pre-activation.

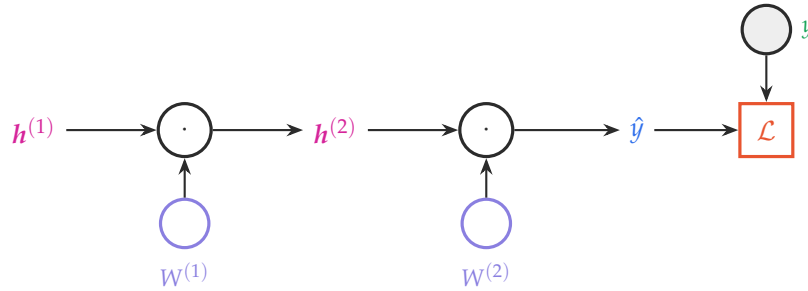
Exercise: let's implement a wacky custom layer. (Colab.)

5. Memory, backprop, and detach

What do we need to store at each layer for backprop? Reading the two rules:

$$\delta^{(k)} = (\delta^{(k+1)} W^{(k+1)\top}) \odot \sigma'(\tilde{h}^{(k)}), \quad \nabla_{W^{(k)}} \mathcal{L} = (h^{(k-1)})^\top \delta^{(k)},$$

each layer needs its own pre-activation $\tilde{h}^{(k)}$ (to evaluate σ') and the input it received, $h^{(k-1)}$. Those are forward quantities, so they have to be kept alive from the forward pass until the backward pass consumes them. That is the memory cost of training, and it is why activations — not parameters — usually dominate it.



Chaining back to the first layer,

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial h^{(2)}} \cdot \frac{\partial h^{(2)}}{\partial h^{(1)}}}_{\text{needs the stored activations}} \cdot \frac{\partial h^{(1)}}{\partial W^{(1)}}.$$

If we **freeze** the lower layers — no gradient required for $W^{(1)}$ — then none of the underbraced factors are needed, and the corresponding **activations** can be discarded as soon as they are used going forward.

`detach()`

`detach` cuts an edge out of the graph: the value is kept, the history is thrown away. Compare:

attached	detached
$y = x \cdot x$	$u = y.\text{detach}()$
$z = y \cdot x$	$z = u \cdot x$
$\frac{\partial z}{\partial x} = \frac{\partial}{\partial x} x^3 = 3x^2$	$\frac{\partial z}{\partial x} = u$

On the left, z knows that y came from x , so both routes contribute. On the right, u is treated as a constant — the same number, no history — so only the explicit x in $z = u \cdot x$ counts. Same forward value, different gradient.

6. When do we actually need a Jacobian?

Three cases, and only the third needs any work:

layer	Jacobian	backward step
linear, $\tilde{\mathbf{h}} = \mathbf{h}\mathbf{W}^{(k)}$	is $\mathbf{W}^{(k)}$ — already have it	matmul, $\delta\mathbf{W}^{(k)\top}$
elementwise, $\mathbf{h} = \sigma(\tilde{\mathbf{h}})$	diagonal	Hadamard, $\odot \sigma'(\tilde{\mathbf{h}})$
outputs coupled	a real matrix	$\delta J^\top$

The test is simple: does output i depend on input j for some $j \neq i$? If not, the Jacobian is diagonal and the whole thing collapses to a multiply.

An example: mean-centering

Here is about the smallest layer that fails that test. Subtract the mean:

$$y_i = x_i - \frac{1}{d} \sum_j x_j.$$

Every output depends on every input, so the Jacobian is not diagonal:

$$J_{ij} = \frac{\partial y_j}{\partial x_i} = \mathbb{1}[i=j] - \frac{1}{d}, \quad J = I - \frac{1}{d} \mathbf{1}^\top \mathbf{1} = \begin{bmatrix} \frac{2}{3} & -\frac{1}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix} \quad (d=3).$$

Forward. $\mathbf{x} = [1 \ 4 \ 1]$ has mean 2, so $\mathbf{y} = [-1 \ 2 \ -1]$.

Backward. Say the error arriving from above is $\delta_{\mathbf{y}} = [1 \ 2 \ 6]$. The route step is the usual one — for each input i , sum (incoming at j) $\times$ (local derivative of y_j with respect to x_i) over everything x_i feeds, which here is every output:

$$\begin{aligned} (\delta_{\mathbf{x}})_i &= \sum_j (\delta_{\mathbf{y}})_j J_{ij} && \text{route step} \\ &= \sum_j (\delta_{\mathbf{y}})_j \left(\mathbb{1}[i=j] - \frac{1}{d} \right) && \text{substitute } J_{ij} \\ &= \underbrace{\sum_j (\delta_{\mathbf{y}})_j \mathbb{1}[i=j]}_{(\delta_{\mathbf{y}})_i} - \frac{1}{d} \sum_j (\delta_{\mathbf{y}})_j && \text{split the sum} \\ &= (\delta_{\mathbf{y}})_i - \text{mean}(\delta_{\mathbf{y}}). \end{aligned}$$

The indicator kills every term of the first sum except $j = i$; the second sum does not depend on i at all, so the same number is subtracted from every component. With $\text{mean}(\delta_{\mathbf{y}}) = (1 + 2 + 6)/3 = 3$:

$$\delta_{\mathbf{x}} = [1 - 3 \ 2 - 3 \ 6 - 3] = [-2 \ -1 \ 3].$$

As a check, multiplying out $\delta_{\mathbf{y}} J$ directly, one component at a time:

$$\begin{aligned} (\delta_{\mathbf{x}})_1 &= 1 \cdot \frac{2}{3} + 2 \cdot \left(-\frac{1}{3}\right) + 6 \cdot \left(-\frac{1}{3}\right) = \frac{2}{3} - \frac{2}{3} - 2 = -2 \\ (\delta_{\mathbf{x}})_2 &= 1 \cdot \left(-\frac{1}{3}\right) + 2 \cdot \frac{2}{3} + 6 \cdot \left(-\frac{1}{3}\right) = -\frac{1}{3} + \frac{4}{3} - 2 = -1 \\ (\delta_{\mathbf{x}})_3 &= 1 \cdot \left(-\frac{1}{3}\right) + 2 \cdot \left(-\frac{1}{3}\right) + 6 \cdot \frac{2}{3} = -\frac{1}{3} - \frac{2}{3} + 4 = 3 \end{aligned}$$

So the backward pass through a centering layer is *itself* a centering: subtract the mean of the incoming error. (Reading the columns of J rather than its rows gives the same numbers, because $J = J^\top$ here.)

Two things worth pointing at:

- **If we had treated this as elementwise** — local derivative 1, pass δ_y straight through — we would have been wrong by the mean, 3 in every slot. The coupling is exactly what the off-diagonals encode.
- δ_x **always sums to zero**, whatever comes in, because every row of J sums to zero. Adding a constant to all of x leaves y unchanged, so the layer is blind in that direction and no gradient can point along it.

This is the centering step inside layer normalization, and softmax has the same character — its outputs compete, its Jacobian has negative off-diagonals, and its rows also sum to zero. Both are layers where a custom backward has to supply something other than a multiply.